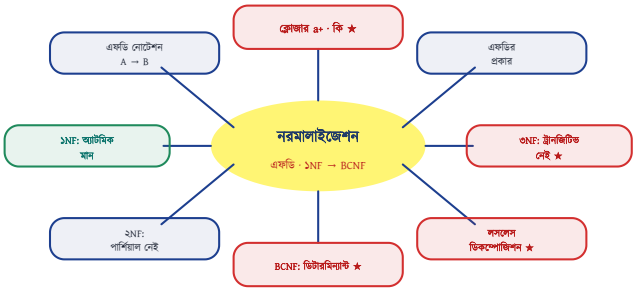
ফাংশনাল ডিপেন্ডেন্সি ও নরমালাইজেশন

এফডি নোটেশন · অ্যাট্রিবিউট ক্লোজার · ১NF → ২NF → ৩NF → BCNF · লসলেস ডিকম্পোজিশন: ১১ পাতায় পরীক্ষার চেলেঞ্জ



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = প্রথম সিঁড়ি

সিলেবাসের যে অংশ

ডেটাবেস সিস্টেমের সিলেবাস লাইনটি পূর্ণ করে: "**Normalization — Process of reducing redundancy and improving data consistency. Steps: 1NF → 2NF → 3NF → BCNF.**" **এই লেসনে ওই অংশটিই নেওয়া হয়েছে।** এই টেবিলগুলো পড়ার রিলেশনাল অ্যালজেবরা ও SQL আগের লেসনে db2-তে এসেছে। পাশের কাজগুলো — ER ও রিলেশনাল মডেল, কনস্ট্রেইন্ট ও ট্রিগার, ফাইল সংগঠন ও স্টোরেজ, প্রাইমারি ও সেকেন্ডারি ইনডেক্স ও B+ ট্রি, কোয়েরি অপটিমাইজেশন — প্রতিটির নিজস্ব জায়গা আছে।

হাই-ইল্ড তালিকা

① A → B: একই A ⇒ একই B
② ট্রিভিয়াল: Y ⊆ X · নন-ট্রিভিয়াল: Y ⊄ X
③ ক্লোজার X^+ = A থেকে বের সব
④ সুপারকি = যেকোনো কি; ক্যান্ডিডেট = ছোটতম
⑤ ১NF: অ্যাটমিক মান, রিপিটিং গ্রুপ নেই
⑥ ২NF: পার্শিয়াল ডিপেন্ডেন্সি নেই
⑦ ৩NF: ট্রানজিটিভ ডিপেন্ডেন্সি নেই
⑧ **BCNF: প্রতিটি ডিটারমিন্যান্ট ক্যান্ডিডেট কি**
⑨ লসলেস ⟺ (R1∩R2)→R1 বা →R2
⑩ লেখায় নরমালাইজ, পড়ায় ডিনরমালাইজ ★
⑪ শেয়ার করা কলাম এক পাশের কি হলে জয়েন লসলেস
⑫ প্রাইম = কোনো না কোনো কির ভেতরে

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি** = মনে রাখার কৌশল · **লাল** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

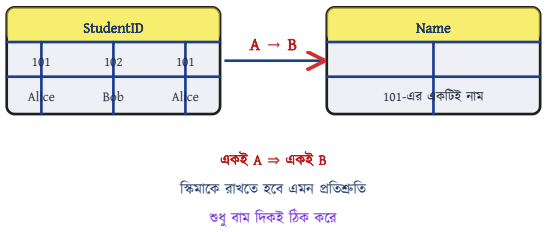
সূচিপত্র

✓ এফডি: নোটেশন ও প্রকার ... পৃ. ২
✓ অ্যাট্রিবিউট ক্লোজার, কি ... পৃ. ৩
✓ ১NF ও ২NF ... পৃ. ৪
✓ ৩NF ও BCNF ... পৃ. ৫
✓ সিঁড়িটি এক টেবিলে ... পৃ. ৬
✓ লসলেস জয়েন, প্রিজার্ভেশন ... পৃ. ৭
✓ নরমালাইজেশন ধাপে ধাপে ... পৃ. ৮
✓ অ্যানোমালি ও ডিনরমালাইজেশন ... পৃ. ৯
✓ রাতের পরীক্ষার পাতা ... পৃ. ১০
✓ অনুশীলনী MCQ + লিখিত ... পৃ. ১১

১ ফাংশনাল ডিপেন্ডেন্সি: A → B আসলে বলে কী

সংজ্ঞা: কোনো রিলেশন R ফাংশনাল ডিপেন্ডেন্সি (FD) X → Y সন্তুষ্ট করে, যদি **X-এর উপর একই মান থাকা যে কোনো দুটো টিপল, Y-এর উপরেও একই মান রাখে**। X হলো **ডিটারমিন্যান্ট**, Y হলো **ডিপেন্ডেন্ট**; A → B পড়ো "A নির্ধারণ করে B"। উদাহরণ: StudentID → Name — ID জানলেই নাম ঠিক, তাই একই ID-এর দুটো সারি কখনো ভিন্ন নাম রাখতে পারে না। FD বলে দেয় রিলেশন *কী বোঝায়*, আজকের সারিগুলো কী আছে তা নয়।

প্রকার	নিয়ম	উদাহরণ	মন্তব্য
ট্রিভিয়াল	Y ⊆ X	(StudentID, Name) → Name	**সবসময় সত্য**
নন-ট্রিভিয়াল	Y ⊄ X	StudentID → Name	আলোচনার যোগ্য এগুলোই
ফুল এফডি	Y-এর জন্য X-এর পুরোটা দরকার	(StudentID, CourseID) → Grade	2NF এটাই দাবি করে
পার্শিয়াল	Y-এর জন্য X-এর অংশই যথেষ্ট	StudentID → StudentName, কি (StudentID, CourseID)	**2NF ভাঙে**
ট্রানজিটিভ	A→B ও B→C মিলে A→C	StudentID → DeptID → DeptName	**3NF ভাঙে**



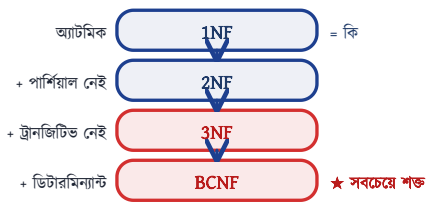
১ম ও ৩য় সারির ID 101, তাই নামও এক

মনে রাখার কৌশল: **"তীর একটি প্রতিশ্রুতি, কাকতালীয় নয়।"** A → B মানে ডেটাবেস কখনোই একই A-সহ দুটি ভিন্ন B রাখতে পারবে না। তাই StudentID একটি কি-ক্যান্ডিডেট, আর "একই ছাত্র, দুটো নাম" একটি **ডিজাইনের ভুল**, ভাগ্যের ঘটনা নয়।

পরীক্ষার ফাঁদ ✍️ দুটি: **১.** (A, B) → A হলো **ট্রিভিয়াল** এবং প্রতিটি রিলেশনে সত্য — কোনো তথ্য নয়, তাই পরীক্ষাও লাগে না। **২.** তীর **একমুখী**: StudentID → Name থেকে Name → StudentID কখনোই বের হয় না, কারণ দুজন ছাত্রের নাম এক হতে পারে।

নরমালাইজেশন: **নরমালাইজেশন** হলো খারাপভাবে সাজানো একটি রিলেশনকে ছোট ভাগে ভেঙে **রিডান্ড্যান্সি কমানো** ও **ডেটার সামঞ্জস্য বাড়ানো**র প্রক্রিয়া, যেখানে প্রতিটি ভাগ একটি **নরমাল ফর্ম** মেনে চলে। সিঁড়িটি ক্রমবর্ধমান: **1NF → 2NF → 3NF → BCNF** — প্রতিটি সিঁড়ি নিচেরটিতে আর একটি শর্ত যোগ করে।

কেন এত কাজ করি: BCNF-এ থাকা রিলেশন প্রতিটি তথ্য **ঠিক এক জায়গায়** রাখে, ফলে তিনটি অ্যানোমালি — আপডেট, ইনসার্ট ও ডিলিট — অদৃশ্য হয়। বিনিময়ে পড়ার সময় বাড়তি জয়েন লাগে।



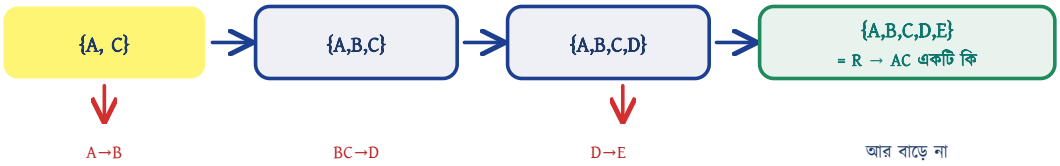
প্রতিটি সিঁড়ি নিচেরটিতে আর একটি শর্ত যোগ করে

২ অ্যাট্রিবিউট ক্লোজার, ক্যান্ডিডেট কি ও সুপারকি

অ্যাট্রিবিউট ক্লোজার X^+: F বারবার ব্যবহার করে X থেকে যে সব অ্যাট্রিবিউট বের হয়, তাদের সেট। **X হলো R-এর একটি কি তখনই, যখন $X^+ = R$।** X দিয়ে শুরু করো, তারপর যেকোনো FD P → Q-এর P আগে থেকেই সেটের ভেতরে থাকলে Q যোগ করো, নতুন কিছু না যোগ হওয়া পর্যন্ত।

```
X⁺ = X
repeat:
  for every FD  P → Q  in F:
      if  P ⊆ X⁺ :  X⁺ ← X⁺ ∪ Q
until  X⁺ stops growing
```

সমাধানকৃত উদাহরণ ১ ✍️ ক্লাসিক ক্লোজার: R(A, B, C, D, E), F = {A→B, BC→D, D→E}। $\{A, C\}^+$ বের করো: শুরু **{A, C}** → A→B প্রযোজ্য → **{A, B, C}** → BC→D প্রযোজ্য → **{A, B, C, D}** → D→E প্রযোজ্য → **{A, B, C, D, E}** = R, তাই **$\{A,C\}^+ = \{A,B,C,D,E\}$** এবং **AC হলো R-এর একটি কি**। পাঁচটি অ্যাট্রিবিউট ঢুকে গেলে F-এর প্রতিটি FD প্রযোজ্য, তাই আর কিছু যোগ হবে না।



ক্লোজারের সিঁড়ি: বাম দিক আগে থেকেই সেটে থাকলে প্রতিটি FD আর একটি অ্যাট্রিবিউট ঢোকায়

সুপারকি · ক্যান্ডিডেট কি · প্রাইম অ্যাট্রিবিউট: যেসব অ্যাট্রিবিউট-সেটের ক্লোজার পুরো R হয় তা **সুপারকি** — উপরে AB তা-ই, কারণ $\{A,B\}^+ = \{A,B,C,D,E\}$। **ক্যান্ডিডেট কি** হলো **মিনিমাল** সুপারকি: তার কোনো উপসেট আর পুরো R বানায় না। কোনো অ্যাট্রিবিউট যদি *কোনো না কোনো* ক্যান্ডিডেট কি-র ভেতরে থাকে, তাকে **প্রাইম** বলে। ৩NF ও BCNF-এর পুরো যুক্তি এই তিনটি শব্দের উপর দাঁড়ায়।

সমাধানকৃত উদাহরণ ২ ✍️ সুপারকির ফাঁদ: R(A, B, C, D), F = {A→B, B→C, C→D}। **A^+:** A → {A,B} → {A,B,C} → {A,B,C,D} = R, তাই A একটি সুপারকি; আর A একটি **একক অ্যাট্রিবিউট** বলে মিনিমাল, ফলে **A-ই ক্যান্ডিডেট কি**। A কোনো FD-এর ডান পাশে নেই, তাই R-এর প্রতিটি কিতেই A থাকতেই হবে। তারপর **AB একটি সুপারকি কিন্তু মিনিমাল নয়** (B সরালে A নিজেই R বানায়), তাই AB **ক্যান্ডিডেট কি নয়**। আর **$B^+ = \{B, C, D\}$** — B থেকে A কখনো আসে না।

পরীক্ষার ফাঁদ ✍️ পুরো প্রশ্নটাই মিনিমালিটি: "ক্যান্ডিডেট কি কোনটি?" মানে **"কোন সেট সব অ্যাট্রিবিউট নির্ধারণ করে?" নয়** — সেট হলো সুপারকি, আর তা সাধারণত একাধিক। তাই দ্বিতীয় ধাপে **মিনিমালিটি** যাচাই করো: একটি অ্যাট্রিবিউট সরিয়ে আবার ক্লোজার বের করো, আর সেট এখনো পুরো R হলে বোঝা সেট বড় ছিল। উপরের R(A,B,C,D)-তে সুপারকি আছে AB, AC, AD, ABC, ABD, ACD, ABCD — আর ক্যান্ডিডেট কি ঠিক **একটিই**, A।

মনে রাখার কৌশল: **"সুপার মানে বড় সেট; ক্যান্ডিডেট মানে তার ভেতরের সবচেয়ে ছোটটা।"** আর **"প্রাইম মানে কোনো না কোনো কির ভেতরে"**, "সব কির ভেতরে" নয় — কি হিসেবে A থাকা R(A,B,C)-তে শুধু A-ই প্রাইম, যদিও A একমাত্র কি। ৩NF ও BCNF-এর পার্থক্য ঠিক প্রাইম/নন-প্রাইম দিয়েই হয়।

৩ প্রথম ও দ্বিতীয় নরমাল ফর্ম

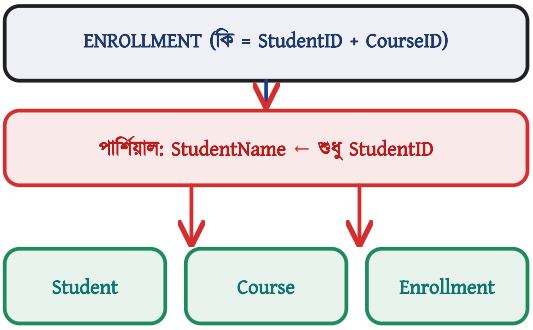
১NF — প্রথম নরমাল ফর্ম: প্রতিটি অ্যাট্রিবিউটে একটিমাত্র **অ্যাটমিক মান** থাকবে। কোনো রিপিটিং গ্রুপ নয়, কোনো তালিকা বা অ্যারে নয়, "এক সারি = অনেক তথ্য" ধরনের কলাম নয়। কোনো মানকে স্বাধীন কয়েকটি মানে ভাঙলে কাজে লাগে না, তাকেই অ্যাটমিক বলে।

১NF-এর আগে	StudentID	Courses
	101	CSE101, MAT201

১NF-এর পরে	StudentID	Course
	101	CSE101
	101	MAT201

পরীক্ষার ফাঁদ: কমা দিয়ে আলাদা করা তালিকা একসাথে **তিনটি** নিয়ম ভাঙছে: সেট অ্যাটমিক নয়, রিপিটিং গ্রুপ, আর এক সারিতে দ্বিতীয় তথ্য লুকিয়ে আছে। ফলে `CSE101`-এ ইনডেক্স করতে স্ট্রিং ছেঁড়ে লাগে, আর "১০১ কতটি কোর্স নিয়েছে?" জিজ্ঞেসের জন্য `LIKE` দিয়ে অনুমান করতে হয়। সমাধানে মাত্র একটি বাড়তি সারি লাগে — তাই সবসময় নাও।

২NF — দ্বিতীয় নরমাল ফর্ম: R তখনই ২NF-এ, যখন সে **১NF**-এ এবং কোনো **নন-প্রাইম** অ্যাট্রিবিউট কম্পোজিট ক্যান্ডিডেট কি-এর **অংশবিশেষের** উপর নির্ভর করে না। কি একক অ্যাট্রিবিউট হলে পার্শিয়াল ডিপেন্ডেন্সি অসম্ভব, তাই ২NF কখনোই শুধু **কম্পোজিট-কি টেবিলে** দাঁড়ায়।



পার্শিয়াল ডিপেন্ডেন্সিই ভাঙটা ঘটায়

সমাধানকৃত উদাহরণ ✍️ ২NF-এর আগে: কি হলো কম্পোজিট **(StudentID, CourseID)**।
`(StudentID, CourseID) → StudentName, Instructor, Grade`
সমস্যা: StudentName শুধু StudentID-এর উপর নির্ভর করে (পার্শিয়াল); Instructor শুধু CourseID-এর উপর নির্ভর করে (পার্শিয়াল)। শুধু → `Grade` পুরো কি লাগে, তাই শুধু সেটিই এখানে থাকে।

Student	Course	Enrollment
StudentID PK	CourseID PK	StudentID PK, FK
StudentName	Instructor	CourseID PK, FK
		Grade

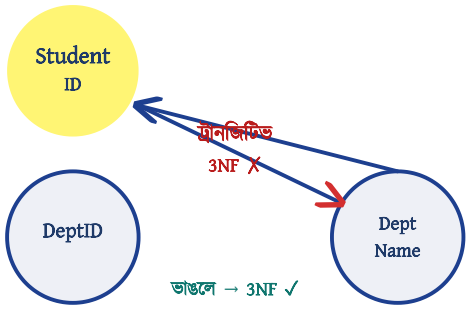
পরীক্ষার ফাঁদ ✍️ দুটি: ১. পার্শিয়াল ডিপেন্ডেন্সি কম্পোজিট কির বিপরীতে চিহ্নিত; একক অ্যাট্রিবিউট কির রিলেশনে ২NF **স্বয়ংক্রিয়** পূরণ হয়, আসল প্রশ্ন তখন ৩NF। ২. ভাগটি **লসলেস** হতে হবে: এখানে R1 ∩ R2 = {StudentID}, আর StudentID → Student (R1-এর পুরোটা), তাই তিনটি টেবিল জয়েন করলে মূল সারিগুলো হুবহু ফিরে আসে।

মনে রাখার কৌশল: **"কলাম ধরে নয়, তীর ধরে ভাঙো।"** প্রথমে FD গুলো আঁকো; যেসব তীরের লেজ কি-এর *অর্ধেক* মাত্র, সেগুলো পার্শিয়াল ডিপেন্ডেন্সি, আর সেই তীরের মাথা নিজের আলাদা টেবিলে চলে যায়। ১NF ভাঙে **মান** অনুযায়ী; ২NF ভাঙে **নির্ভরতা** অনুযায়ী।

৪ তৃতীয় ও বয়েস-কড নরমাল ফর্ম

৩NF — তৃতীয় নরমাল ফর্ম: R তখনই ৩NF-এ, যখন সে **২NF**-এ এবং কোনো নন-প্রাইম অ্যাট্রিবিউট কির উপর **ট্রানজিটিভ**ভাবে নির্ভর করে না: প্রতিটি FD X → A-এ, যেখানে A নন-প্রাইম, X অবশ্যই সুপারকি। সহজ কথায়: **কির অংশ নয় এমন কোনো অ্যাট্রিবিউট অন্য অ্যাট্রিবিউটের উপর নির্ভর করতে পারে না**।

সমাধানকৃত উদাহরণ ✍️ আসল ৩NF লঙ্ঘন: **আগে** — `Student(StudentID, Name, DeptID, DeptName)`, কি **StudentID**: `StudentID → Name, DeptID`, তারপর `DeptID → DeptName` তাই `StudentID → DeptID → DeptName`। DeptID কি-এর অংশ **নয়**, অথচ DeptName তার উপর ঝুলে আছে — নন-কি অ্যাট্রিবিউটের উপর ট্রানজিটিভ নির্ভরতা। **পরে** — `Student(StudentID, Name, DeptID·FK)` এবং `Department(DeptID, DeptName)`। এখন DeptName যত ছাত্র থাকুক বিভাগ প্রতি একবারই সংরক্ষিত হয়।



৩NF-এ কথা হয় এই তীর নিয়েই

BCNF — বয়েস-কড নরমাল ফর্ম: R তখনই BCNF-এ, যখন **প্রতিটি** নন-ট্রিভিয়াল FD X → Y-এর জন্য X একটি **সুপারকি**। পরীক্ষার সবচেয়ে ছোট রূপ: **প্রতিটি ডিটারমিন্যান্ট একটি ক্যান্ডিডেট কি হতে হবে**। এটি ৩NF-এর চেয়ে কঠোর — ৩NF এই শর্তটি তখনই চায় যখন ডিপেন্ডেন্ট *নন-প্রাইম*।

সমাধানকৃত উদাহরণ ✍️ ৩NF যা সারাতে পারে না: ধরো `R(CourseID, Semester, Instructor)`, যেখানে `(CourseID, Semester) → Instructor` এবং `Instructor → CourseID`। প্রতিটি পাশ হাতে আলাদা করে দেখো: না **CourseID** একা, না **Semester** একা পুরো রিলেশন নির্ধারণ করে, তাই ডিটারমিন্যান্ট (CourseID, Semester)-ও সুপারকি নয়। কিন্তু Instructor একটি প্রাইম অ্যাট্রিবিউট, তাই রিলেশনটি **২NF**-ই মেনে চলে — BCNF লঙ্ঘন। **BCNF সমাধান:** ভাঙো — `Instructor(CourseID, Instructor)` (কি Instructor) এবং `Offering(CourseID, Semester)`।

পরীক্ষার ফাঁদ ✍️ তিনটি: **১.** ৩NF বলে "প্রতিটি *নন-প্রাইম* অ্যাট্রিবিউট কির উপর নির্ভর করে"; BCNF বলে "প্রতিটি *ডিটারমিন্যান্ট* কি"। শৃঙ্খল যখন একটি **প্রাইম** অ্যাট্রিবিউটের উপর দিয়ে যায়, তখন রিলেশন ৩NF মেনে চলেও BCNF লঙ্ঘন করে। **২.** একক অ্যাট্রিবিউট কির ক্ষেত্রে কোনো পার্শিয়াল ডিপেন্ডেন্সি থাকে না, তাই রিলেশন ২NF-এ থেকে ৩NF লঙ্ঘন করতে পারে। **৩. BCNF সিঁড়ির শেষ নয়:** মাল্টিভ্যালুড ডিপেন্ডেন্সির জন্য ৪NF আছে।

মনে রাখার কৌশল: **"১NF একটি মান, ২NF পুরো কি, ৩NF কোনো ঘুরপথ নয়, BCNF প্রতিটি তীর কি থেকে।"** BCNF-কে *বাম দিকের* নিয়ম হিসেবে পড়ো: F-এর প্রতিটি FD দিয়ে যাও এবং প্রশ্ন করো "এই ডিটারমিন্যান্ট কি তো?" যেকোনো একটির উত্তর না হলে সেই রিলেশন ভাঙো।

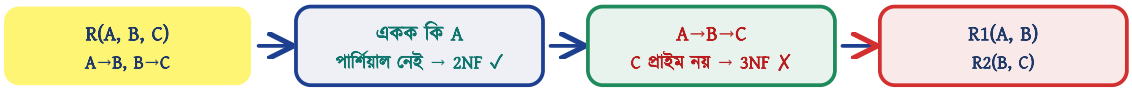
পরীক্ষার ফাঁদ ✍️ আরও দুটি: **৪. সুপারকি ও ক্যান্ডিডেট কি গুলিয়ে ফেলা** যায় — BCNF-এ "determinant is a candidate key" বলতে গিয়ে অনেকে পুরো সুপারকির কথা বলে ফেলে; পরীক্ষায় খোঁজা হয় **মিনিমাল** সেটটিকে। আর **৫.** "৩NF-এ ট্রানজিটিভ নির্ভরতা নেই" — কথাটা যত বড়, তত ভুল ধরার ঝুঁকি; মনে রাখো শর্তটি হলো **নন-প্রাইম** গন্তব্যের উপর যাওয়া ট্রানজিটিভ নির্ভরতা।

৫ সিঁড়িটি এক টেবিলে — আর কতদূর ওঠা যায়

নরমাল ফর্ম	শর্ত	সহজ কথায়
১NF	অ্যাটমিক মান, রিপিটিং গ্রুপ নেই	এক সেলে একটি তথ্য
২NF	১NF + **পার্শিয়াল ডিপেন্ডেন্সি নেই**	কম্পোজিট কি-এর অর্ধেকের উপর কিছু ঝোলে না
৩NF	২NF + **ট্রানজিটিভ ডিপেন্ডেন্সি নেই**	নন-কি অ্যাট্রিবিউটের উপর কিছু ঝোলে না
BCNF	৩NF + **প্রতিটি ডিটারমিন্যান্ট একটি ক্যান্ডিডেট কি**	প্রতিটি তীর কি থেকে শুরু হয়

সমাধানকৃত উদাহরণ ✍️ "সর্বোচ্চ নরমাল ফর্ম": R(A, B, C), F = {A→B, B→C}, আর A-ই একমাত্র ক্যান্ডিডেট কি।
কি বের করা। A কোনো ডান পাশে নেই, তাই প্রতিটি কিতে A থাকবেই। A^+ = {A, B, C} = R, আর A একটি অ্যাট্রিবিউট, তাই A-ই **একমাত্র ক্যান্ডিডেট কি** ও একমাত্র **প্রাইম** অ্যাট্রিবিউট।
২NF? কি একটি **একক** অ্যাট্রিবিউট, তাই পার্শিয়াল ডিপেন্ডেন্সি সম্ভবই নয়। **R হলো ২NF-এ।**
৩NF? A→B→C ট্রানজিটিভ: B→C-এ B সুপারকি নয়, C-ও প্রাইম নয়। তাই R ৩NF লঙ্ঘন করে, BCNF-ও। **উত্তর: ২NF।**
ডিকম্পোজিশন: R1(A, B) ও R2(B, C)।

সমাধানকৃত উদাহরণ ✍️ কম্পোজিট কি হলে:
R(StudentID, CourseID, Grade, StudentName), কি (StudentID, CourseID)।
StudentID → StudentName — লেজ কি-এর অর্ধেক মাত্র, তাই **পার্শিয়াল**।
ধাপ ৩ ধাপেই শেষ: পার্শিয়াল ডিপেন্ডেন্সি থাকলে রিলেশন ২NF-এই আটকে থাকে, ট্রানজিটিভ বা ডিটারমিন্যান্ট পরীক্ষা করার দরকার হয় না। ভাঙো Student(StudentID, StudentName) ও Enrollment(StudentID, CourseID, Grade)-এ।



উত্তর ২NF — একটিই উত্তর, দুটো ধাপ মিলিয়ে নয়

একক কি হলে ২NF স্বয়ং, আর ট্রানজিটিভ থাকলে ৩NF ভাঙে

যেকোনো "সর্বোচ্চ নরমাল ফর্ম" প্রশ্নের উত্তর: চারটি ধাপ, এই ক্রমে: **১.** ক্যান্ডিডেট কি বের করো (ক্লোজার ও মিনিমালিটি); **২.** প্রাইম অ্যাট্রিবিউটগুলো লিখো; **৩.** কি কম্পোজিট হলে **পার্শিয়াল** ডিপেন্ডেন্সি খোঁজো — থাকলেই ১NF-এ থেমে যাও; **৪.** না থাকলে নন-কি অ্যাট্রিবিউটের উপর **ট্রানজিটিভ** ডিপেন্ডেন্সি খোঁজো — থাকলে ২NF; না থাকলেও কোনো ডিটারমিন্যান্ট কি না হলে ৩NF; নইলে BCNF।

গোটা সিঁড়ির মনে রাখার কৌশল: **"অ্যাটমিক, পুরো কি, কি-র বাইরে নয়, আর প্রতিটি তীর কি থেকে।"** ১NF মানের ব্যাপার, ২NF কি-এর অংশের ব্যাপার, ৩NF নন-কি ঘুরপথের ব্যাপার, BCNF ডিটারমিন্যান্টগুলোর নিজের ব্যাপার। প্রতিটি সিঁড়ি হলো "নিচেরটা + আর একটি শর্ত", তাই BCNF-এ থাকা রিলেশন নিচের তিনটাতেই থাকে, উল্টোটা কখনো নয়।

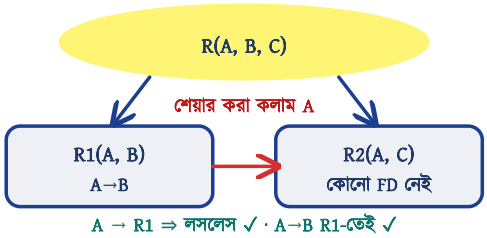
পরীক্ষার ফাঁদ ✍️: ট্রানজিটিভ ডিপেন্ডেন্সি দেখেই অনেকে "৩NF" লিখে ফেলেন, ভুলে যান যে ৩NF কেবল **নন-প্রাইম** গন্তব্যের উপর যাওয়া ট্রানজিটিভ নির্ভরতাই নিষেধ করে। আর A একা যখনই R বানিয়ে ফেলে তখন AB-কে ক্যান্ডিডেট কি বলে ফেলা হয় — পরীক্ষা হয় **মিনিমালিটি** নিয়ে, আকার নিয়ে নয়।

৬ ডিকম্পোজিশন: লসলেস জয়েন ও ডিপেন্ডেন্সি প্রিজার্ভেশন

একটি রিলেশন ভাঙা তখনই বৈধ নরমালাইজেশন, যখন সেটি **লসলেস** (কোনো তথ্য বানানো বা নষ্ট হয় না) এবং **ডিপেন্ডেন্সি-প্রিজার্ভিং** (জয়েন ছাড়াই FD মানা যায়)।

লসলেস-জয়েনের নিয়ম: R → R1, R2-তে জয়েন তখনই **লসলেস**, যখন
(R1 ∩ R2) → R1 অথবা (R1 ∩ R2) → R2
পড়ো: দুই ভাগের **যে অ্যাট্রিবিউটগুলো শেয়ার করে**, তারা অন্তত একটি ভাগের কি হতে হবে।

ডিপেন্ডেন্সি প্রিজার্ভেশন: ডিকম্পোজিশন তখনই **ডিপেন্ডেন্সি-প্রিজার্ভিং**, যখন F-এর R1 ও R2-তে প্রক্ষিপ্ত অংশের ইউনিয়ন ঠিক F-ই হয় — বাস্তবে তখন, যখন **F-এর প্রতিটি FD সম্পূর্ণভাবে একটি ভাগের ভেতরে থাকে** এবং জয়েন ছাড়াই সেখানে মানা যায়।



শেয়ার করা অ্যাট্রিবিউট একটি পুরো পাশ ঠিক করে

সমাধানকৃত উদাহরণ ✍ দুই অংশের উত্তর: R(A, B, C), F = {A→B}, ভাঙা হয়েছে R1(A, B) ও R2(A, C)-এ।
লসলেস? R1 ∩ R2 = {A}, আর A→B মানে A→R1, তাই শেয়ার করা অ্যাট্রিবিউট A হলো R1-এর কি। নিয়মের প্রথম বিকল্প সত্য → **লসলেস**: প্রতিটি মূল টিপলের একটাই A-মান, তাই ঠিক একটি R1 সারি ও একটি R2 সারি জোড়া হবে।
প্রিজার্ভড? F-এর একমাত্র FD A→B সম্পূর্ণভাবে R1-এর ভেতরে, তাই জয়েন ছাড়াই সেখানে মানা যায় → **ডিপেন্ডেন্সি-প্রিজার্ভিং**।

পরীক্ষার ফাঁদ ✍ লসি উদাহরণ: একই R(A, B, C), F = {A→B} নিয়ে ভুলভাবে ভাঙো R1(A, C) ও R2(B, C)-এ। এখন R1 ∩ R2 = {C}, আর C কোনো ভাগই ঠিক করে না, তাই নিয়ম ভাঙে: জয়েন **লসি** — R1 থেকে C = 3 কে R2-এর C = 3 এর সঙ্গে জোড়া করলে (3, 3) সারি তৈরি হয়, যা R-এ কখনো ছিল না। ভাগ এমনভাবে করতে হবে যাতে শেয়ার করা অ্যাট্রিবিউট একটি পাশে **কি** হয়।

সমাধানকৃত উদাহরণ ✍ লসলেস, কিন্তু প্রিজার্ভড নয়: R(A, B, C), F = {A→B, B→C}, ভাঙা হয়েছে R1(A, B) ও R2(A, C)-এ।
লসলেস? R1 ∩ R2 = {A}, আর A→B মানে A→R1, তাই প্রথম বিকল্প সত্য — জয়েন লসলেস।
প্রিজার্ভড? A→B ঠিক আছে R1-এর ভেতরে, কিন্তু B→C কোনো একটি ভাগের ভেতরে নেই — মানতে জয়েন লাগবে। তাই ডিপেন্ডেন্সি-প্রিজার্ভিং নয়। ডেটাবেস B→C একা কোথাও মানাতে পারবে না।

পরীক্ষার ফাঁদ ✍ লসলেস ≠ প্রিজার্ভড: দুটো পরীক্ষা আলাদা প্রশ্নের উত্তর, আর একটি ডিকম্পোজিশন একটায় পাশ আর অন্যটায় ফেলতে পারে। লসলেস হয়েও A→B মানতে **জয়েন** বসাতে হতে পারে, তখন ডেটাবেস কনস্ট্রেন্ট মানতে পারে না। আবার প্রতিটি FD প্রিজার্ভ করেও জয়েন লসি হতে পারে — তখন নিঃশব্দে ডেটা নষ্ট হয়। দুটো উত্তর আলাদা করে দাও।

মনে রাখার কৌশল: **"লসলেস জিজ্ঞেস করে ওভারল্যাপ নিয়ে, প্রিজার্ভেশন জিজ্ঞেস করে তীর নিয়ে।"** ওভারল্যাপ: শেয়ার করা কলাম একটি পুরো পাশ ঠিক করতে হবে। তীর: প্রতিটি FD একটি পাশের ভেতরে থাকতে হবে। কোনো পরীক্ষার জন্যই ডেটা বা সারির সংখ্যা লাগে না — **শুধু FD থেকেই উত্তর বের হয়**।

৭ সম্পূর্ণ নরমালাইজেশনের উদাহরণ, ধাপে ধাপে

StudentID	Name	DeptID	DeptName	CourseID	CourseName	Grade
101	Alice	10	CSE	C101	DBMS	A
101	Alice	10	CSE	C102	Algorithms	B

এই এক টেবিলের সমস্যা কী: এটি আগে থেকেই ১NF-এ — প্রতিটি সেলে একটিমাত্র মান। কিন্তু কি কম্পোজিট (StudentID, CourseID), আর টেবিলে তিনটি তথ্য পুনরাবৃত্ত: **Alice** দুবার লেখা, **10 / CSE** দুবার লেখা, আর কোর্সের নাম প্রতি ছাত্রের জন্য একবার করে লেখা হতো। তাই ছাত্রের নাম ও বিভাগের নাম প্রতিটি কয়েক সারিতে আছে, আর ওই সারিগুলোর যেকোনো একটি অন্যগুলোর সঙ্গে দ্বিমত করতে পারে।

যে তিনটি অ্যানোমালি হয়: **আপডেট** — Alice-এর নাম বদলাতে হলে 101 যেসব সারিতে আছে সবগুলোতে করতে হবে; একটি বাদ পড়লেই ডেটাবেসে ছাত্র ১০১-এর দুটো নাম থাকবে। **ইনসার্ট** — একেবারে নতুন ও কোনো ছাত্র নেই এমন বিভাগ রেকর্ড করা যায় না। **ডিলিট** — Alice-এর একমাত্র এনরোলমেন্ট সারিটি মুছলে তার নাম *এবং* বিভাগ ১০ আছে এই তথ্যও হারিয়ে যায়।

ধাপ ১ — ১NF ✍️: আগে থেকেই পূরণ: প্রতিটি কলামে একটি অ্যাটমিক মান, আর গ্রেড কোনো একক ছাত্রের নয়, জোড়া (ছাত্র, কোর্স)-এর। কিছু করার নেই।

ধাপ ২ — ২NF ✍️ কি (StudentID, CourseID) কম্পোজিট, আর তিনটি নন-কি অ্যাট্রিবিউট কি-এর **অর্ধেকের** উপর নির্ভর করে: `StudentID → Name`, `CourseID → CourseName` ও `StudentID → DeptID` — সবই পার্শিয়াল; অথচ `(StudentID, CourseID) → Grade` ফুল।

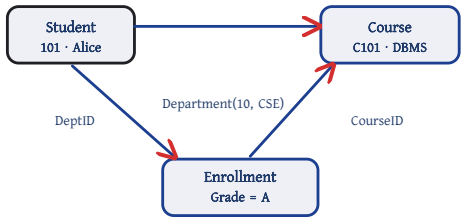
ধাপ ৩ — ৩NF ✍️ আসল অবশিষ্ট লঙ্ঘন হলো `Student(StudentID, Name, DeptID, DeptName)`-এর ট্রানজিটিভ শৃঙ্খল `StudentID → DeptID → DeptName`: DeptID কি-এর অংশ **নয়**, অথচ DeptName তার উপর ঝোলে। DeptName বেরিয়ে নিজস্ব Department টেবিলে যাওয়াই ঠিক যেটা এটা সরায় — ৩NF ধাপের পুরো বিষয় এটাই।

Student	Name	DeptID
101	Alice	10

Department	DeptName
10	CSE

Course	CourseName
C101	DBMS

Enrollment	Grade
101	A



একটি তথ্য, এক জায়গায়

কি রেখা দেওয়া আছে; DeptID এবং Enrollment-এর দুটো ID-ই উপরের টেবিলে ফরেন কি।

চূড়ান্ত ডিজাইন, ও যা আমরা পাই: চারটি রিলেশন: **Student(StudentID, Name, DeptID)** · **Department(DeptID, DeptName)** · **Course(CourseID, CourseName)** · **Enrollment(StudentID, CourseID, Grade)**। গ্রেড থাকে তার নিজের জোড়ার সঙ্গে, বিভাগের নাম **একবার** সংরক্ষিত, কোর্সের নামও **একবার**। ছাত্রের নাম বদলাতে এখন **একটিমাত্র** সারির আপডেট লাগে, খালি বিভাগ ঢোকানো যায় কোনো ছাত্র বানিয়ে না দিয়েই, আর এনরোলমেন্ট মুছলে বিভাগও চলে যায় না।

পরীক্ষার ফাঁদ ✍️ ভাগ কি লসলেস ছিল?: ছয় ধাপেই ভাগ ছিল, এবং প্রতিবার ওই শর্ত ছিল যে শেয়ার করা অ্যাট্রিবিউট সংশ্লিষ্ট ভাগগুলোর একটির কি। ২NF-এর ভাগে শেয়ার করা ছিল StudentID, যা Student-এর কি; ৩NF-এর ভাগে DeptID, যা Department-এর কি। তাই জয়েন করলে মূল টেবিলের প্রতিটি সারি হুবহু ফিরে আসে, কোনো সারি বাড়ে না বা কমে না।

৮ অ্যানোমালি, ডিনরমালাইজেশন ও ট্রেড-অফ

তিনটি অ্যানোমালির নাম: একই তথ্য কয়েক জায়গায় রাখার ফলেই যে ভুল আপডেট ঘটে, তাকে **অ্যানোমালি** বলে। মাত্র তিনটি প্রকার আছে, আর লিখিত প্রশ্নে সাধারণত তিনটির নামই চায়।

আপডেট অ্যানোমালি ✍️: একটি তথ্য **সব কপিতে** বদলাতে হয়, নইলে রিলেশন নিজেই অসংগত হয়ে যায়। Alice-এর নাম বদলাতে 101 যেসব সারিতে আছে সবগুলোতে করতে হবে; একটি বাদ পড়লে একই ছাত্রের দুটো নাম থেকে যায়।

ইনসার্ট অ্যানোমালি ✍️: একটি বৈধ তথ্য **সংরক্ষণই করা যায় না**, কারণ কি এমন একটি সঙ্গী তথ্য চায় যা এখনো নেই। কোনো ছাত্র নেই এমন নতুন বিভাগের সঙ্গে রাখার মতো কোনো StudentID নেই।

ডিলিট অ্যানোমালি ✍️: একটি সারি মুছে দিলে সেটির সঙ্গে কোনো সম্পর্ক নেই এমন তথ্যও নিঃশব্দে চলে যায়। Alice-এর একমাত্র এনরোলমেন্ট সারিটি মুছলে তার নাম ও বিভাগ ১০ আছে— এই তথ্যও মুছে যায়।



ভাঙাটা কেন করা সার্থবহ

নরমালাইজেশনের সুবিধা: ১. কম রিডান্ড্যান্সি, তাই কম ডিস্ক ও একসঙ্গে রাখার কম কপি। ২. ভালো ইন্টিগ্রিটি — প্রতিটি তথ্যের একটিই ঘর, তাই একটি আপডেটেই পুরো আপডেট। ৩. কম আপডেট, ইনসার্ট ও ডিলিট অ্যানোমালি। ৪. **ছোট ও অর্থবহ টেবিল**, তাই ইনডেক্স ও জয়েন পরিকল্পনা সস্তা।

ডিনরমালাইজেশন — ইচ্ছাকৃত উল্টো চাল: **ডিনরমালাইজেশন** হলো নরমালাইজ করার পর **ইচ্ছাকৃতভাবে** রিডান্ড্যান্সি ফিরিয়ে আনা, যাতে **পড়া** দ্রুত হয়। নরমালাইজড স্কিমায় রিপোর্টের একটি সারি বের করতে তিনটি টেবিল জয়েন লাগে; ফ্যাক্ট টেবিলে বিভাগের নাম কপি করলে সেই জয়েন একটি স্ক্যানে হয়ে যায়। এটি সচেতন বাণিজ্য — **একবার লিখুক, লক্ষবার পড়ুক** — আর এতে তিনটি অ্যানোমালি ফিরে আসে।

পরীক্ষার ফাঁদ ✍️ দিকটার প্রশ্ন: "কোনটি ভালো?"-এর কোনো নির্দিষ্ট উত্তর নেই — নির্ভর করে কাজের ধরনের উপর। অল্প ছোট ইনসার্ট, আপডেট ও ডিলিট এবং ডেটাবেস-চালিত ইন্টিগ্রিটি থাকলে OLTP-র জন্য নরমালাইজ করো; বিশাল পড়ার ভলিউম ও মাঝে মাঝে ব্যাচ রিলোড থাকলে **রিপোর্টিং / ডেটা ওয়্যারহাউসিং**-এর জন্য ডিনরমালাইজ করো। "ডিনরমালাইজেশন সবসময় কর্মক্ষমতা বাড়ায়" — এটি **মিথ্যা**: এটি পড়া দ্রুত করে আর প্রতিটি লেখা ধীর করে।

পরীক্ষার ফাঁদ ✍️ ঝুঁকিগুলো: কপি করা তথ্য দুই জায়গায় বাস্তব, তাই দুই জায়গাতেই আপডেট দিতে হয় — আর একটি বাদ পড়লেই সেটি অ্যানোমালি। তাই ডিনরমালাইজেশন করা হয় শুধু তখনই, যখন পড়া:লেখার অনুপাত সত্যিই অনেক বেশি, এবং অ্যাপ্লিকেশন সেই অতিরিক্ত কপি ঠিক রাখার দায়িত্ব নিতে পারে। সাধারণ উপায়: সময়-সময় পুরো ফ্যাক্ট টেবিল ব্যাচ রিবিল্ড করা।

এই অধ্যায়ের বাকি অংশ কোথায়: **এই টেবিলগুলো পড়া** — সিলেকশন, প্রজেকশন, ছয় জয়েন, অ্যাগ্রিগেট — আগের লেসনে db2। **রিলেশনাল ও ER মডেল, কনস্ট্রেইন্ট** — db1। **ফাইল সংগঠন, ইনডেক্স, B+ ট্রি, কস্ট-বেসড অপটিমাইজেশন** — কোন ইনডেক্স কোন কোয়েরি সস্তা করে → db4। **ট্রানজ্যাকশন, ACID, আইসোলেশন, কনকারেন্সি, রিকভারি** → db5।

রাতের পরীক্ষার পাতা

তথ্যপাতা:

$X \to Y$: একই X ⇒ একই Y

X ডিটারমিন্যান্ট, Y ডিপেন্ডেন্ট

ট্রিভিয়াল: $Y \subseteq X$ · নন-ট্রিভিয়াল: $Y \not\subset X$

পার্শিয়াল: X-এর অংশে · ট্রানজিটিভ: A→B, B→C

X^+ = X থেকে বের সব

X একটি কি তখনই, যখন $X^+ = R$

সুপারকি = যেকোনো কি

ক্যান্ডিডেট কি = মিনিমাল সুপারকি

প্রাইম = কোনো কির ভেতরে

১NF: অ্যাটমিক মান

২NF: + পার্শিয়াল ডিপেন্ডেন্সি নেই

৩NF: + ট্রানজিটিভ ডিপেন্ডেন্সি নেই

BCNF: + প্রতিটি ডিটারমিন্যান্ট কি

অ্যানোমালি: আপডেট, ইনসার্ট, ডিলিট

লসলেস ⟺ $(R1 \cap R2) \to R1$ বা $\to R2$

প্রিজার্ভড ⟺ প্রতিটি FD এক ভাগে

R(A,B,C,D,E)-তে $\{A,C\}^+$ = পাঁচটিই

A→B→C→D-এ R(A,B,C,D)-র কি একা A

A→B→C, কি A হলে মাত্র ২NF

ভাঙো: R1(A, B), R2(B, C)

শেয়ার করা অ্যাট্রিবিউট এক পাশের কি হলে লসলেস

প্রতিটি FD এক ভাগে থাকলে প্রিজার্ভড

লেখায় নরমালাইজ, পড়ায় ডিনরমালাইজ

BCNF-এর পরে ৪NF আছে (মাল্টিভ্যালুড ডিপেন্ডেন্সি)

MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. A → B: A-তে মিললে B-তেও মিলবে
2. ট্রিভিয়াল FD: $Y \subseteq X$ — সবসময় সত্য, মজার নয়
3. $X^+ = R$ মানে X একটি কি
4. সুপারকি = যেকোনো কি; ক্যান্ডিডেট = ছোটতমটা
5. প্রাইম অ্যাট্রিবিউট = কোনো ক্যান্ডিডেট কির ভেতরে
6. ১NF মানে প্রতি সেলে একটি অ্যাটমিক মান
7. ২NF কম্পোজিট কির ক্ষেত্রেই কাজ করে
8. ৩NF নন-কি অ্যাট্রিবিউটের ট্রানজিটিভ নির্ভরতা নিষেধ করে
9. BCNF: প্রতিটি ডিটারমিন্যান্ট একটি ক্যান্ডিডেট কি
10. লসলেস ⟺ $(R1 \cap R2) \to R1$ বা $(R1 \cap R2) \to R2$
11. লসলেস ওভারল্যাপ দেখে, প্রিজার্ভেশন তীর দেখে
12. অ্যানোমালি তিনটি: আপডেট, ইনসার্ট ও ডিলিট
13. কম্পোজিট কি থাকলে পার্শিয়াল থাকলেই ২NF-এ থামো
14. ৪NF আছে, তবে "সর্বোচ্চ" মানে BCNF

সবচেয়ে বড় ফাঁদগুলো: **সুপারকি**কে ক্যান্ডিডেট কি বলা · **মিনিমালিটি** যাচাই না করা · প্রাইম মেম্বারশিপ দেখা ছাড়া "৩NF" বলা · একক অ্যাট্রিবিউট কিতে ২NF দেখানো · তীর উল্টো দিকে পড়া · ২NF-এর "আগে" টেবিলে **কম্পোজিট কি** না লেখা · **লসলেস-জয়েন** না যাচাই করেই ডিকম্পোজিশন নিরাপদ বলা · লসলেস ও প্রিজার্ভেশন গুলিয়ে ফেলা।



প্রতি সিঁড়িতে একটি শর্ত

সব কৌশল একসাথে: তীর একটি প্রতিশ্রুতি · প্রথমে মান অনুযায়ী, তারপর তীর অনুযায়ী একটি তথ্য এক টেবিলে · কলাম ধরে নয়, তীর ধরে ভাঙো · সুপার মানে বড় সেট, ক্যান্ডিডেট মানে ভেতরের সবচেয়ে ছোটটা · অ্যাটমিক, পুরো কি, কি-র বাইরে নয়, প্রতিটি তীর কি থেকে · লেখায় নরমালাইজ, পড়ায় ডিনরমালাইজ।

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

১. একটি রিলেশন First Normal Form (1NF)-এ থাকে যদি:
ক) প্রতিটি অ্যাট্রিবিউটে কেবল অ্যাটমিক মান থাকে
খ) এতে কোনো পার্শিয়াল ডিপেন্ডেন্সি নেই
গ) এতে কোনো ট্রানজিটিভ ডিপেন্ডেন্সি নেই
ঘ) প্রতিটি ডিটারমিন্যান্ট একটি ক্যান্ডিডেট কি

২. R(A, B, C, D)-এ A→B, B→C, C→D। R-এর ক্যান্ডিডেট কি কোনটি?
ক) B
খ) AB
গ) A
ঘ) CD

৩. R(A, B, C, D, E)-এ A→B, BC→D, D→E। ক্লোজার $\{A, C\}^+$ কী?
ক) {A, B, C}
খ) {A, B, C, D}
গ) {A, C, E}
ঘ) {A, B, C, D, E}

৪. R(A, B, C)-এ A→B, B→C, আর A একমাত্র ক্যান্ডিডেট কি। R সর্বোচ্চ কোন ফর্মে?
ক) কেবল 1NF
খ) 2NF
গ) 3NF
ঘ) BCNF

৫. F = {A→B} সহ R(A, B, C)-কে R1(A, B) ও R2(A, C)-তে ভাঙা হলো। কোনটি সঠিক?
ক) লসি কিন্তু ডিপেন্ডেন্সি-প্রিজার্ভিং
খ) লসলেসও নয়, প্রিজার্ভিংও নয়
গ) লসলেস কিন্তু প্রিজার্ভিং নয়
ঘ) লসলেস-জয়েন ও ডিপেন্ডেন্সি-প্রিজার্ভিং

৬. কি (StudentID, CourseID) রেখে Enrollment(StudentID, CourseID, Grade, StudentName, Instructor)-এ কোন FD পার্শিয়াল?
ক) StudentID→StudentName
খ) (StudentID, CourseID)→Grade
গ) CourseID→Instructor
ঘ) তিনটিই

৭. একটি রিলেশন BCNF-এ থাকবে কেবল যখন:
ক) এতে ট্রানজিটিভ ডিপেন্ডেন্সি নেই
খ) প্রতিটি ডিটারমিন্যান্ট একটি ক্যান্ডিডেট কি
গ) এতে পার্শিয়াল ডিপেন্ডেন্সি নেই
ঘ) প্রতিটি অ্যাট্রিবিউট অ্যাটমিক মান রাখে

৮. R কে R1 ও R2-তে ভাঙলে জয়েন লসলেস হবে যখন:
ক) R1 ∩ R2 খালি
খ) R1 ও R2 দুটোই 3NF-এ
গ) (R1 ∩ R2)→R1 বা (R1 ∩ R2)→R2
ঘ) প্রতিটি ডিপেন্ডেন্সি প্রিজার্ভ হয়

৯. ডিনরমালাইজেশন ইচ্ছাকৃতভাবে ব্যবহার করা হয় কেন?
ক) রিডান্ড্যান্সি ও অ্যানোমালি সরাতে
খ) রিডান্ড্যান্সি মেনে নিয়ে পড়ার কোয়েরি দ্রুত করতে
গ) ফরেন-কি কনস্ট্রেইন্ট স্বয়ংক্রিয়ভাবে মানাতে
ঘ) আপডেট অ্যানোমালি কমাতে

১০. R(A, B, C)-এ A→B, B→C আর A একমাত্র ক্যান্ডিডেট কি হলে কতটি প্রাইম অ্যাট্রিবিউট আছে?
ক) 1
খ) 2
গ) 3
ঘ) 0

উত্তরমালা → ১-ক · ২-গ · ৩-ঘ · ৪-খ · ৫-ঘ · ৬-ক · ৭-খ · ৮-গ · ৯-খ · ১০-ক

লিখিত প্রশ্ন: ফাংশনাল ডিপেন্ডেন্সির সংজ্ঞা দাও এবং পাঁচটি প্রকারের নাম দাও — ট্রিভিয়াল বনাম নন-ট্রিভিয়াল পার্থক্য করে। F = {A→B, BC→D, D→E} সহ R(A, B, C, D, E)-এর $\{A, C\}^+$ বের করো এবং AC কি কিনা বলো। A→B, B→C সহ R(A, B, C)-এ, যেখানে A একমাত্র ক্যান্ডিডেট কি, সর্বোচ্চ নরমাল ফর্ম বলো ও ডিকম্পোজিশন দাও। F = {A→B} সহ ওই R-কে R1(A, B) ও R2(A, C)-তে ভাঙা হলে জয়েন লসলেস কিনা ও ডিপেন্ডেন্সি প্রিজার্ভ কিনা পরীক্ষা করো। ১০ নম্বর

১ **সংজ্ঞা (২)** — X → Y তখনই সত্য যখন X-এ মিলে যাওয়া যে কোনো দুটো টিপল Y-এও মেলে; X হলো ডিটারমিন্যান্ট। **ট্রিভিয়াল:** Y ⊆ X, যেমন (A, B)→A। **নন-ট্রিভিয়াল:** Y ⊄ X, যেমন StudentID→Name। **ফুল:** Y-এর জন্য X-এর পুরোটা দরকার। **পার্শিয়াল:** X-এর অংশবিশেষই যথেষ্ট। **ট্রানজিটিভ:** A→B ও B→C মিলে A→C। ২

২ **ক্লোজার (২)** — {A, C}; A→B এ B যোগ করে → {A, B, C}; BC→D এ D যোগ করে → {A, B, C, D}; D→E এ E যোগ করে → **{A, B, C, D, E} = R**, তাই **AC একটি ক্যান্ডিডেট কি**। ২

৩ **সর্বোচ্চ নরমাল ফর্ম (২)** — A কোনো ডান পাশে নেই এবং একটি অ্যাট্রিবিউট, তাই এটিই একমাত্র ক্যান্ডিডেট কি ও একমাত্র প্রাইম অ্যাট্রিবিউট। একক কি মানে পার্শিয়াল ডিপেন্ডেন্সি নেই, তাই R **2NF**-এ। কিন্তু A→B→C ট্রানজিটিভ, আর B→C-এ ডিটারমিন্যান্ট সুপারকি নয় ও C প্রাইম নয়, তাই 3NF ব্যর্থ: উত্তর **2NF**, ডিকম্পোজিশন **R1(A, B)** ও **R2(B, C)**। ২

৪ **লসলেস (২)** — R1 ∩ R2 = {A}, আর A→B মানে A→R1, তাই শেয়ার করা অ্যাট্রিবিউট R1-এর কি: জয়েন **লসলেস**। ২

৫ **প্রিজার্ভেশন ও নিয়ম (২)** — একমাত্র FD A→B সম্পূর্ণভাবে R1-এর ভেতরে, তাই জয়েন ছাড়াই মানা যায়: **ডিপেন্ডেন্সি-প্রিজার্ভিং**। সাধারণ নিয়ম: লসলেস ⟺ (R1∩R2)→R1 বা (R1∩R2)→R2; প্রিজার্ভিং ⟺ F-এর প্রক্ষিপ্ত অংশের ইউনিয়ন F। ২

পরীক্ষার আগে ✓

- ✓ সংজ্ঞা ও পাঁচটি এফডি প্রকার লিখতে পারি
- ✓ X^+ বের করে ক্যান্ডিডেট কি খুঁজতে পারি
- ✓ সুপারকি আর ক্যান্ডিডেট কি আলাদা করতে পারি
- ✓ চারটি নরমাল ফর্মের শর্ত বলতে পারি
- ✓ ডিকম্পোজিশন লসলেস কিনা যাচাই করতে পারি
- ✓ অ্যানোমালি ও ডিনরমালাইজেশনের নাম বলতে পারি